Introduction
Transfer-Optimization (FIFO)

INFORM

Agenda

- Introduction
- Constraints
- Transformations
- Optimization Approach
- Additional Parameters
- Conclusion

INFORM

INTRODUCTION

Overview

Objective: Schedule all deliveries with as little delay as possible

Secondary goals:
- Avoid unnecessary trips
- Comply with First-in-First-out (*FiFo*)-rules where first vehicle gets next load

General: Minimize the number of open deliveries and the delay of deliveries.

Including various constraints such as special treatment for the early morning (*First Round*).

Approach

Heuristic approach with two-step procedure, since exact methods are too computationally intensive

1. Plant Dispatch

- Work on plant basis, i.e., vehicles on site are used to deliver orders on site (principal plant is used if available), and comply with FiFo-rule

➔ Delays are possible

2. Transformations

- Exchange work (deliveries) and/or resources (vehicles) between plants

Results:
Schedule with information for each delivery (loading times, plant, vehicle) → *Proposals*

CONSTRAINTS

INFORM

Input Data

Plants

Vehicles/trucks

Order

- Quantity (min, max)
- Continuous delivery flow and *load-spacing*
- Customer

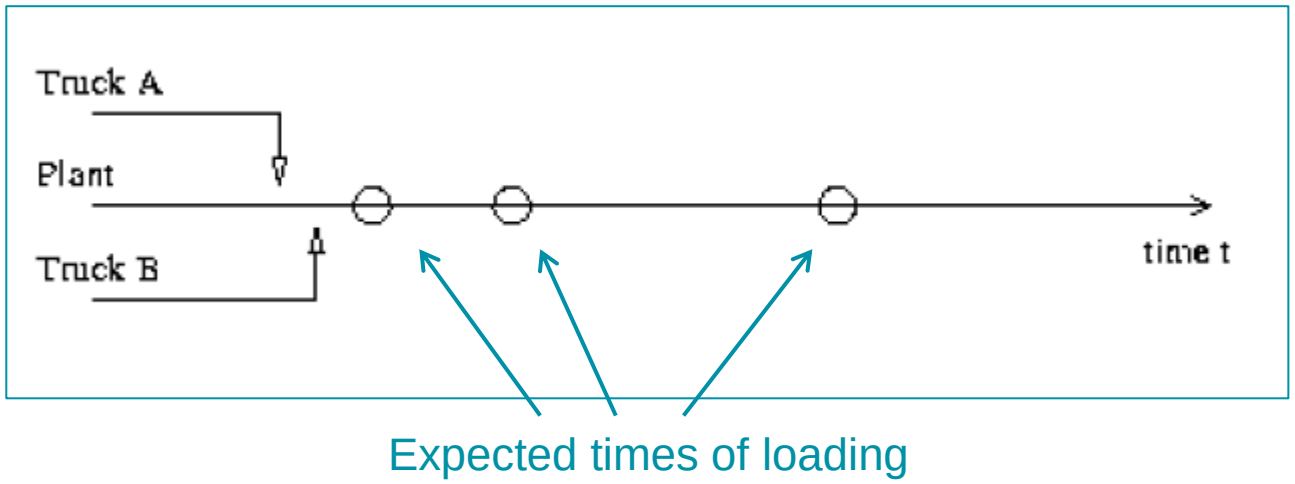
(Assigned) deliveries with delivery time intervals

Principal plant (usually nearest plant) and **alternative plants**

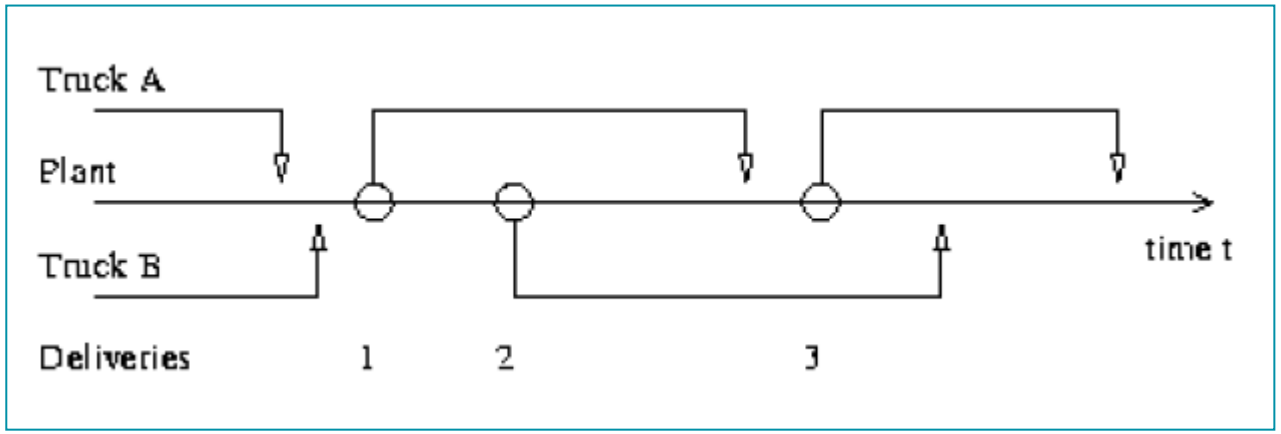
Constraints

- Shift and working times for single shift functionality and planning for one day of a *Dispatch Group* (DG)
- FiFo-rule at concrete plants with exceptions
- First Round-rules: Deviation from planned start time in the morning is considered (fairness)
- Priorities and delivery time window of orders (defined via start of first delivery and delivery flow)
- Continuous delivery flow with specified quantity per time unit and predecessor orders with offset
- Delays of deliveries are allowed – non-linear cost function
- *Early loading* is allowed
- Properties/exclusions of vehicles/shifts, orders und plants
- Vehicle capacities and priorities and *Plusloads*
- Opening hours of plants, parking and customer locations
- Maximum capacity per concrete plant
- Number of loading slots at plants (*legs*) and alternative plants
- Cleaning durations and fixed and variable unloading durations

FiFo-Rule

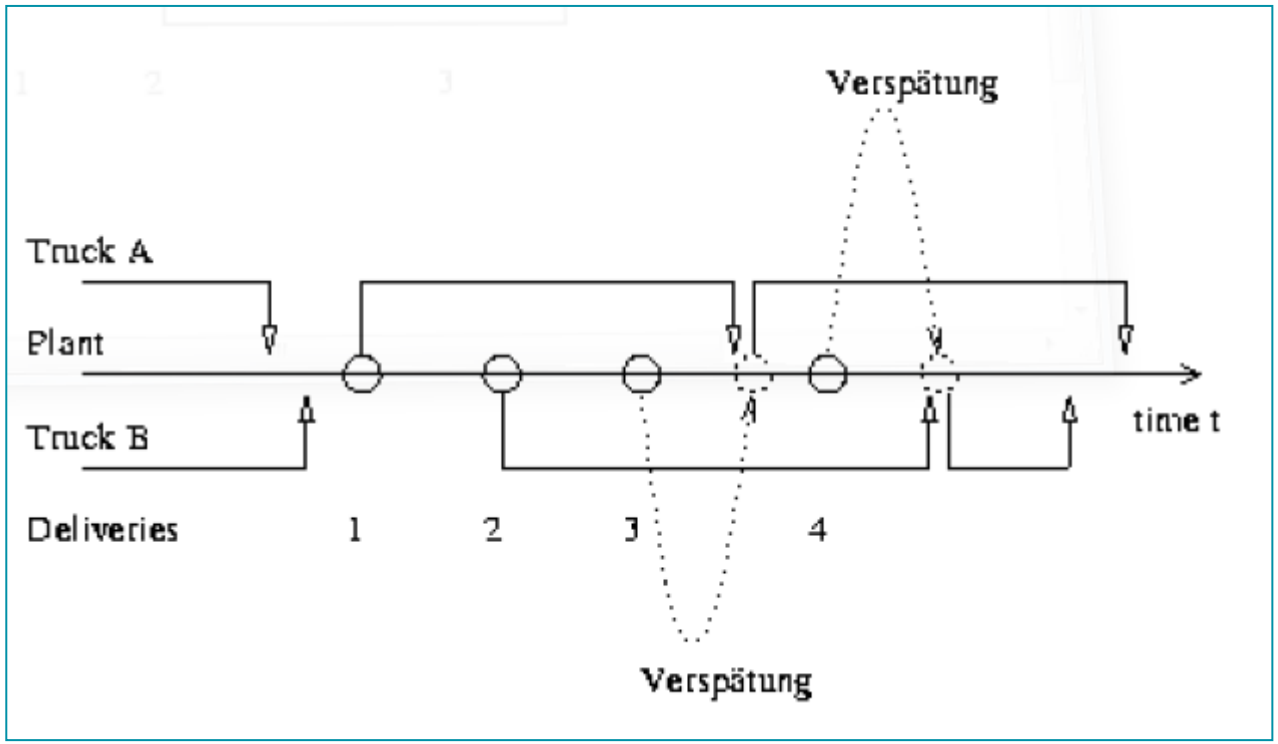


Expected times of loading



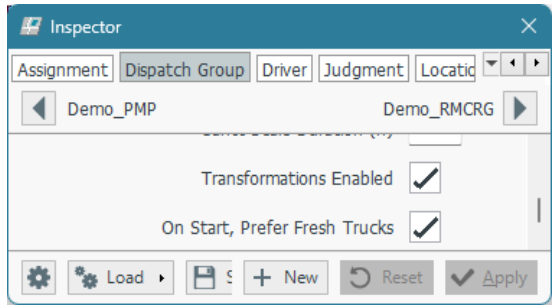
- First vehicle at plant gets next delivery
- FiFo-rule very restrictive
  → Fast assignment of vehicles and deliveries
- Approach considers vehicles that arrived in the past at the plants → idle time
- .. and vehicles that arrive in the future
- Exceptions of FiFo-rule
  - Properties (vehicle and delivery do not match!)
  - First Round-rules
  - High-priority-orders
  - …

FiFo-Rule



- Situation with delay
- Note: All following deliveries of the same order must be postponed → Load-spacing
- See GUI
  → Delivery Overview

First Round and Morning Roster



Exception rule for first day‘s work assignment

Prefer vehicles that have not worked before – DG-parameter `thePreferNotYetWorked`

Dispatcher determines *Morning Roster* (MR) at plant that defines sequence of planned vehicles

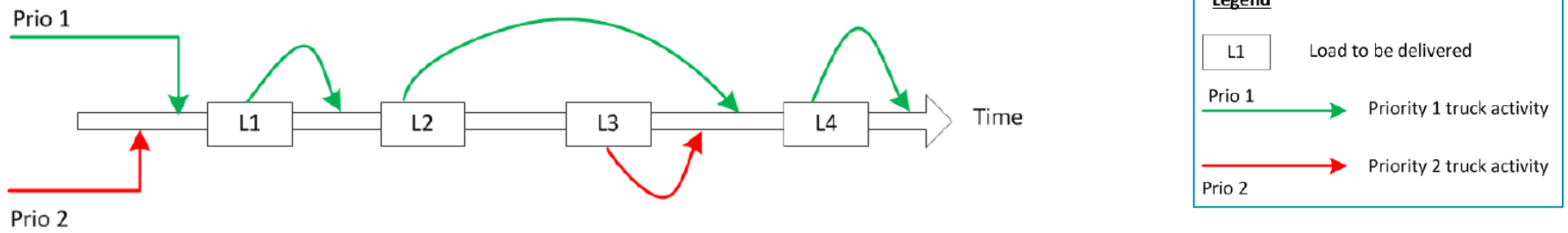
- Approach uses start times from roster (false) or own start times (true) with `roster.fifo_does_the_roster`
- Switch for role of roster group and number is DG-parameter `theContinuousRosterFlag`

Preplanning provides assigned sign on (*SAS*) and considers the Roster

Realtime-Optimization:

- Planning of vehicles with SAS
- No Proposals without SAS until vehicle is signed on
- Late sign on, i.e., after SAS, results in preference of late vehicle
- Early sign on, i.e., prior to SAS, does not result in preference → Exception: Delay can be avoided!

Vehicle Priorities



Vehicles with higher priority 1 are preferred over vehicles with priority 2, if

- the delay by using a higher prioritized vehicle is not exceeded over that of a lower prioritized,
- the MR is not violated, i.e., prioritizing of vehicles does not work for first delivery,
- a higher prioritized vehicle arrives until the expected load time of a lower prioritized vehicle plus the difference between the DG-parameters `theTruckHoldingTimePrio1` and `theTruckHoldingTimePrio2`

Vehicle priorities can be configured by `theHaulierCostPriority` at the resource

Early Loading

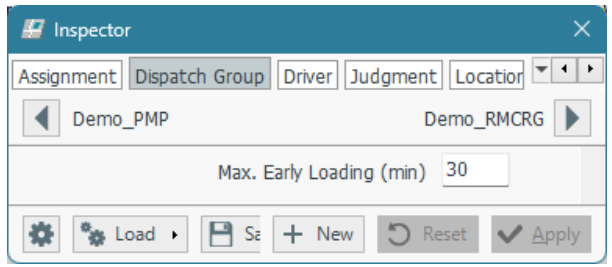
Plant capacity = Number of deliveries per hour or quantity per time unit

Overlap of two deliveries d_1,d_2:

(1) Delay d_2
(2) Prefer d_1 ➔ „Early loading“

Optimization tries (2) first

Maximum early loading can be configured by DG-parameter `theDefaultShelfLife`, i.e., the earliest start loading timestamp is the left side of the delivery time interval minus the travel from plant to customer site, the load duration, and the defined shelf time



Restrictions:

- Arrival of proposed vehicle
- Start time of optimization
- Fixed deliveries

INFORM

Properties and Plant Dispatch

Properties and exclusions work for the following combinations:

- Order – shift
- Unload location – shift
- Load location – shift → can be configured by `optim.fifoCheckPlantExclusion`

Plant Dispatch: Allows for vehicles working from home plant

- Deliveries are scheduled at principal plant if available
- If principal plant is not available for loading, e.g., by opening hours, the nearest alternative is used
  “Only principal plant usage” can be configured by `optim.fifoAllowInitialMultiplanting`

TRANSFORMATIONS

Approach

(Optional) second phase (actual optimization) enabled by DG-parameter `theEnableTrafos`

Work (deliveries) and resources (vehicles) are exchanged between plants

Objective: Reduce number of remaining and late deliveries, …

… but avoid *excessive* travel time

Approach (iterative):

- Plant Dispatch for each plant
- Calculation of (lateness-)costs for each plant
- Determination of Pre-Selects, i.e., pairs of plants (acceptor, provider) for exchange
- Generation of predefined number of transformations (selects)
- Evaluation of transformations
- Execution of *best* transformation

→ Assessment by costs

Assessment of Transformations

Cost are artifical and usually time-based

- Assessment of overall situation by one transformation t:

  ➔ Difference of cost at Provider + Difference of cost at Acceptor + Cost of t (additional travel)

- Select transformation with highest improvement of total cost

- Repeat until (1) no more transformation with improvement can be found, i.e., all t with cost ≥ 0, or (2) the maximum number of transformations is done

- *Iterative approach will be described in more detail in the next section*

Costs

Plant costs consist of four terms

- Delay! (delivery)
- Early Loading (delivery)
- Overtime (vehicle)
- Idle-Times (vehicle)

DG-parameters:

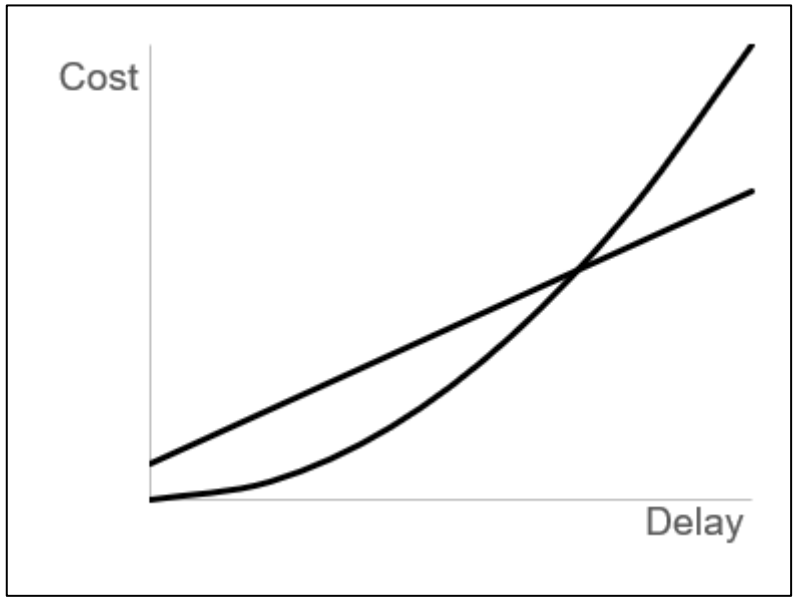
`theScaleEarlyLoad`	Weight for early loads
`theScaleIdle`	Weight for idle times at plant
`theScaleOvertime`	Weight for overtime at plant
`theCapIdle`	Minimum for idle time at plant
`theCapOvertime`	Minimum for overtime at plant

Delay Costs

Delay: Difference between proposed load time and end of load time-interval

- Quadratic and linear parts of cost function with d : delay:

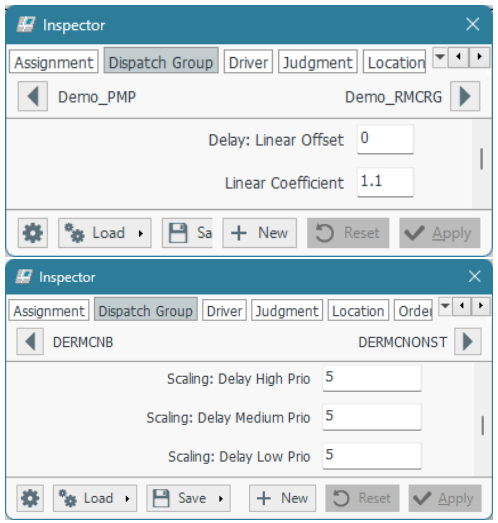
$$\min(m \cdot d + n, c * d^2)$$



DG-parameters:

m	n
theScaleDelay	theScaleDelayOffset

Priority of order	c
0, 1, 5, 6	theScaleSquare1
2	theScaleSquare2
3	theScaleSquare3

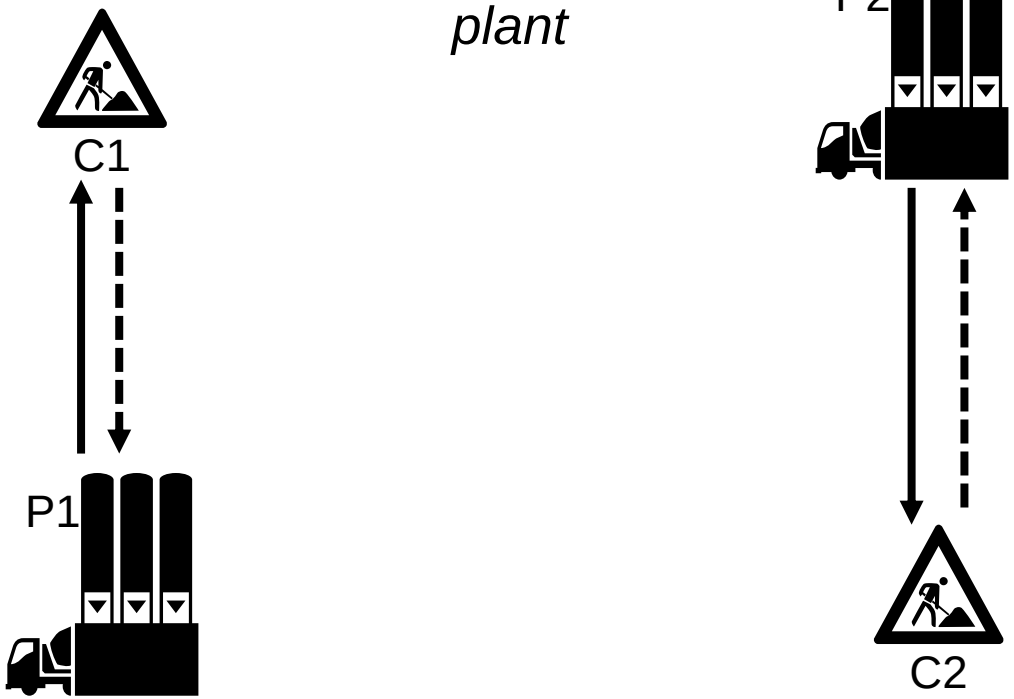


Plusloads have priority 0 while the default for all orders is priority 2

Types of Transformation

Plant Dispatch

Delivery from principal or closest available plant



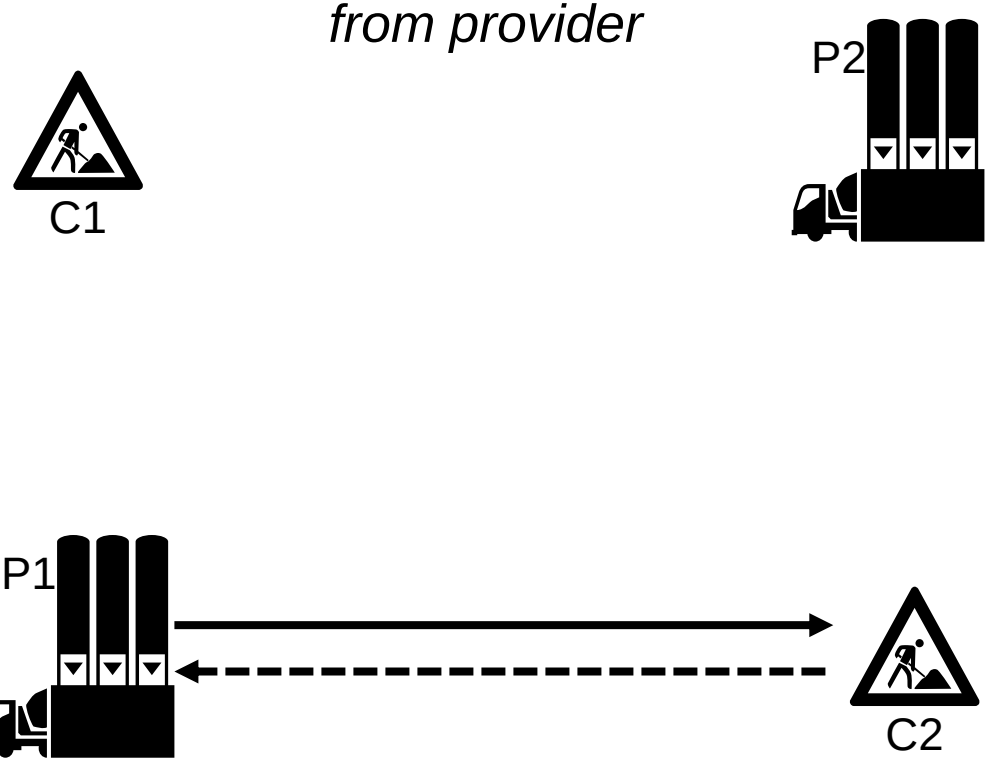
Types of Transformation

Plant Dispatch

Four Transformations:

- Alternative Plant

Exchange of work by direct delivery from provider

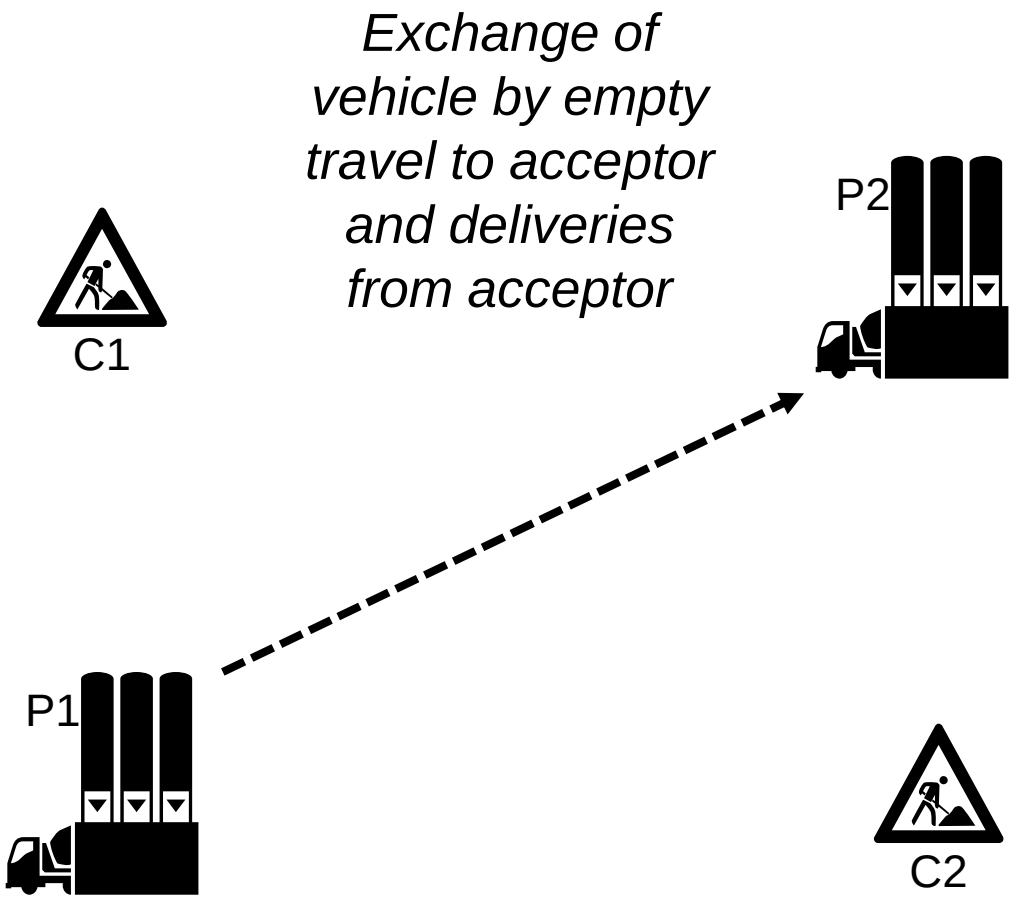


Types of Transformation

Plant Dispatch

Four Transformations:

- Alternative Plant
- Approach from Plant

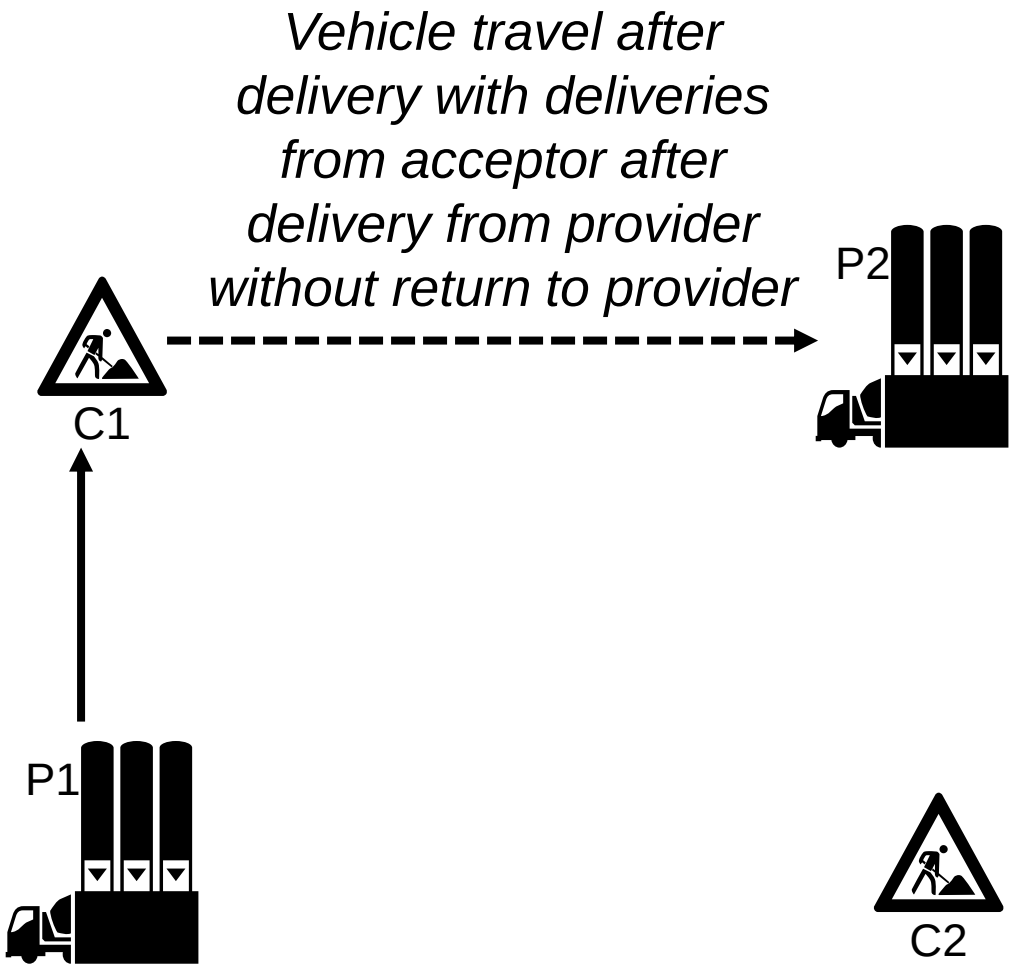


Types of Transformation

Plant Dispatch

Four Transformations:

- Alternative Plant
- Approach from Plant
- Approach from Customer



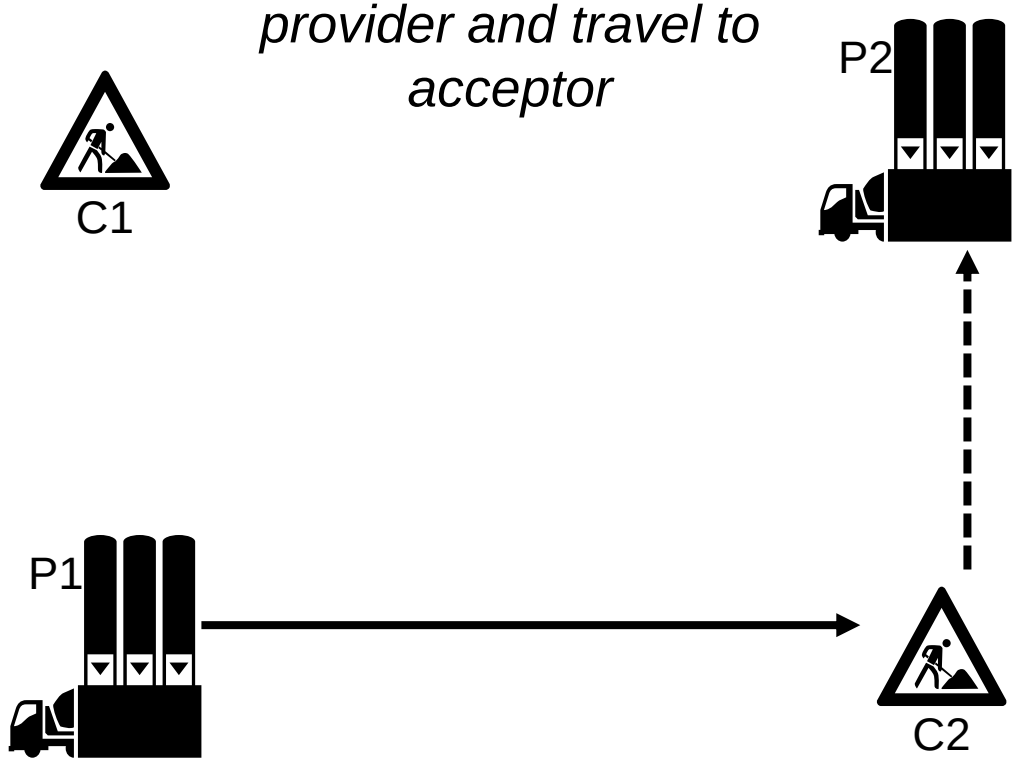
Types of Transformation

Plant Dispatch

Four Transformations:

- Alternative Plant
- Approach from Plant
- Approach from Customer
- Approach by Delivery

Vehicle travel and alternative plant by direct delivery from provider and travel to acceptor



INFORM

Types of Transformation

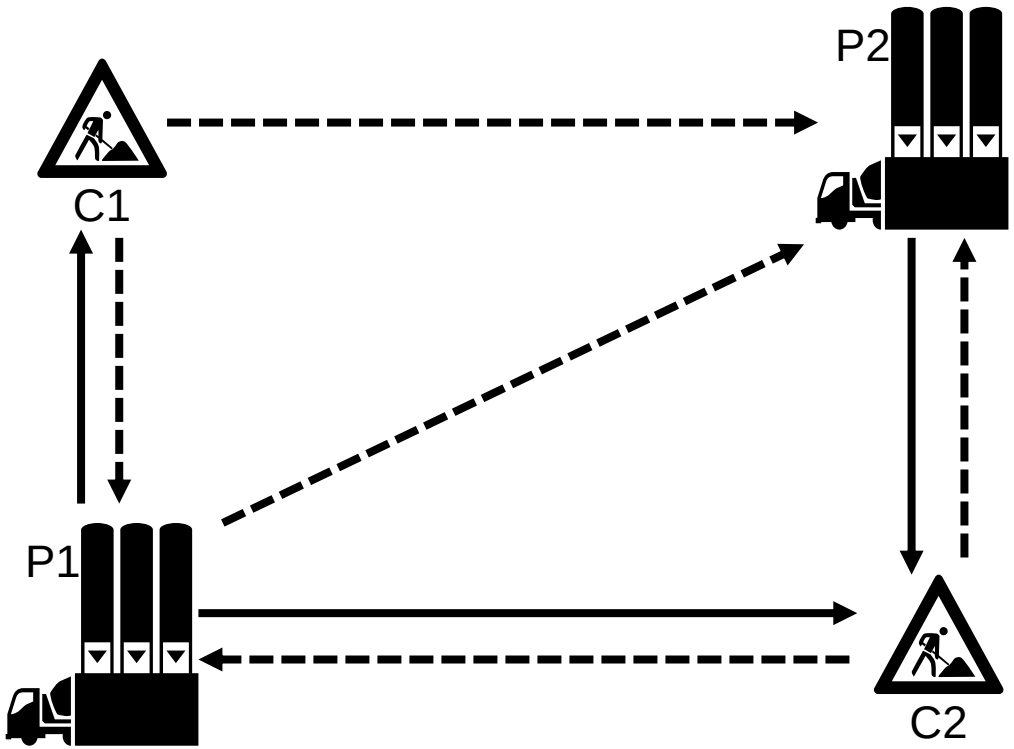
Plant Dispatch

Four Transformations:

- Alternative Plant (AP)
- Approach from Plant (AfP)
- Approach from Customer (AfC)
- Approach by Delivery (AbD)

Exchange resources (vehicles) between plants

Exchange work (deliveries) between plants



Cost Weighting

Cost of a transformation t can be weighted by configuration:

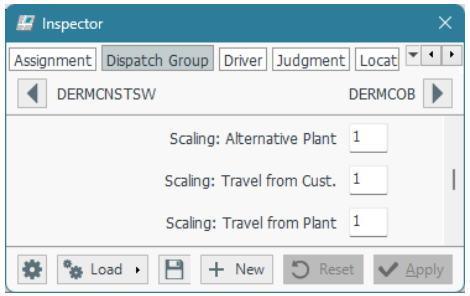
Difference of cost at provider	+ Difference of cost at acceptor	+ Cost of t
↑	↑	↑
Potentially more delay at provider	Potentially less delay at acceptor	Additional travel

→ Cost must be negative for improvement by transformation t

Additional travel is caused by alternative plant that is further away (AP, AbD) and/or sending vehicles to the acceptor plant (AfP, AfC, AbD).

Each of the transformation types can be weighted to prefer or downgrade relatively to other types

Cost Weighting - Parameters



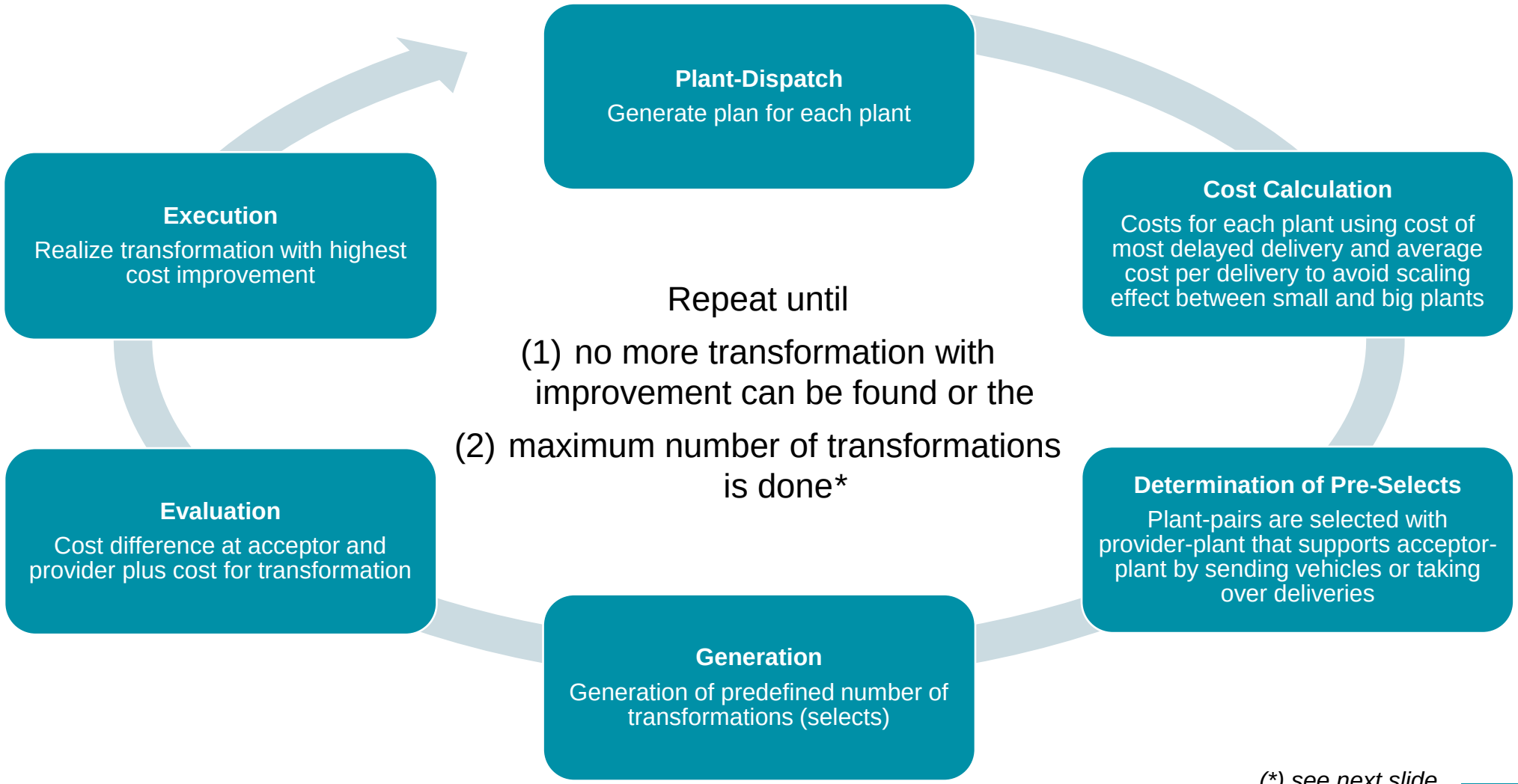
DG-parameters to weight cost of transformation t depending on type

`theScaleAlternativePlant p`	Weight for AP-transformations: $2p \cdot$ additional travel by t
`theReturnHomeWeight r`	Weight for additional distance to home location of vehicle
`theScaleApproachFromCustomer p`	Weight for AfC- and AbD-transformations $p \cdot$ (additional return to plant travel by t $+ r \cdot$ additional return to home travel by t) [AfC] $p \cdot$ (additional approach to unload location travel by t $+ r \cdot$ additional return to home travel by t) [AbD]
`theScaleApproachFromPlant p`	Weight for AfP-transformations $p \cdot$ (additional return to plant travel by t $+ r \cdot$ additional return to home travel by t)

DG	`thePlantDispatchOnly`	Forbid all transformations where plants are changed (AP, AbD)
`optim`	`fifoForbidSomeTrafos`	Comma-separated list with string entries AbD, AfC, AfP, AP (see types of transformations)

OPTIMIZATION APPROACH

Iterative Optimization Approach



(*) see next slide

Number of Transformations

(1) No more transformation with improvement can be found, i.e., cost of all transformations is positive

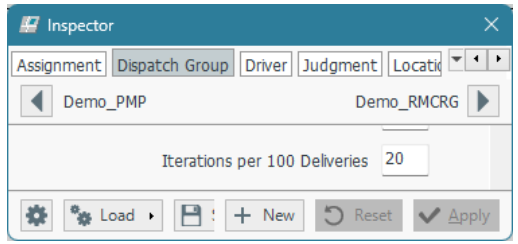
- Force to make at least one transformation even it does not improve by `optim.fifoAtLeastOneTrafo`

(2) Maximum number of transformations is done

- DG-parameter `theMaxTrafosPer100Deliveries` `p` results in maximum of `20` or

  $\frac{p \cdot n}{100}$, n: number of deliveries at plant,

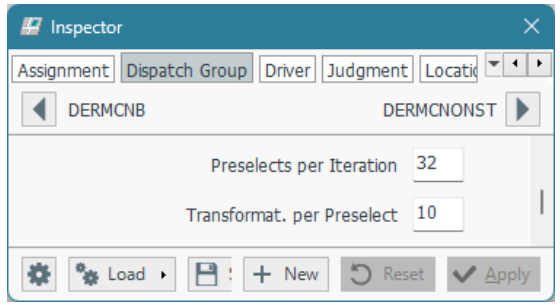
  but overwritten by `fear.MaxNofFifoTransformations`, if > 0



Pre-Selects

Calculate cost for each pair of plants by

Cost of most expensive delivery at **provider**
+ **Travel** between plants
- **Cost** of most expensive delivery at **acceptor**



Weight of travel duration between plants (provider, acceptor) in case of transformation can be configured by DG-parameter `thePreSelectTruckDurationFactor`

Select best pairs → Find pairs where acceptor has high cost and provider can support = Pre-Selects
DG-parameter for number of selected Pre-Selects: `thePreSelectsPerIteration`

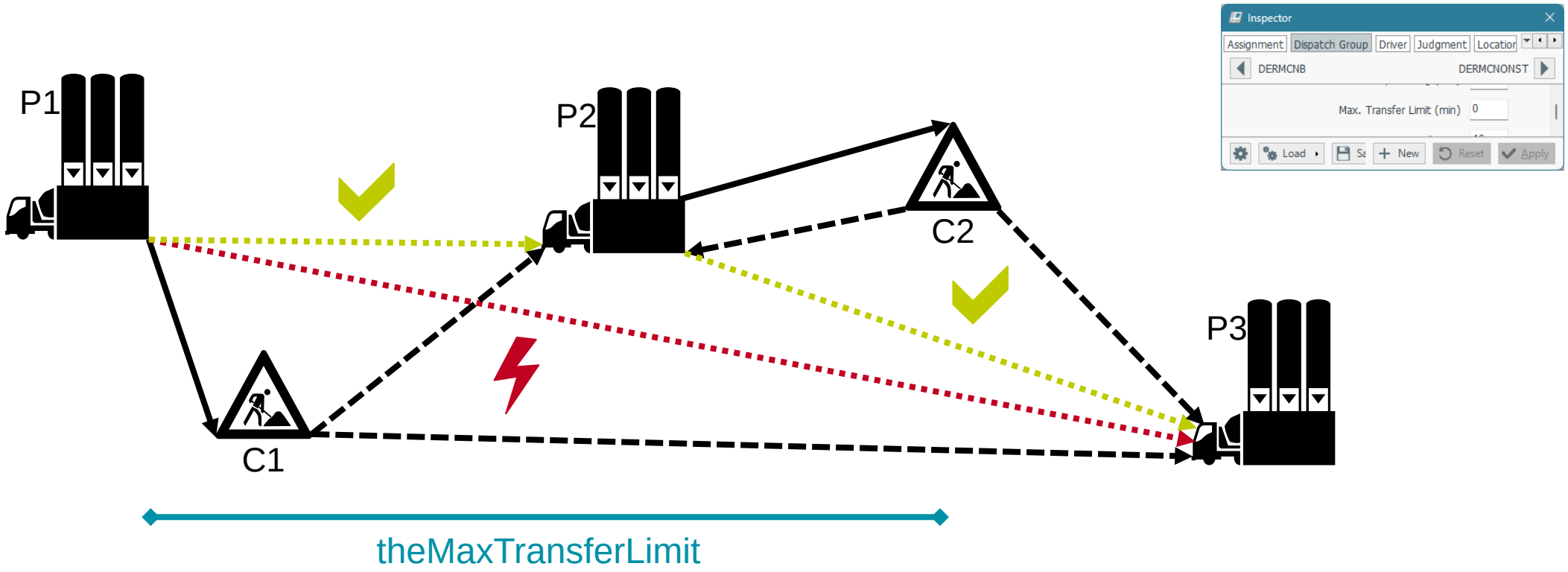
Determine specific transformations for each Pre-Select, i.e., pre-defined number *n* of transformations per Pre-Select = Selects

- Parameter for number of Selects *n* per Pre-Select: `theTransformationsPerPreSelect`

Selects

Deliveries at acceptor are checked if they can be moved to provider, i.a., provider is AP

- Maximum distance for transformation defined by DG-parameter `theMaxTransferLimit`
- Maximum distance to return location defined by `optim.fifoMaxReturnDistance`



Delivery Selects

2/3 n delivery-Selects (AP)

- Check each delivery at acceptor if it can be moved to provider
- Estimate cost/effort of each moved delivery
  - Delay, time since start of optimization, rare properties, high priorities
  - No exchange of deliveries of First Round without delay
- Select best *2/3 n* transformations

Vehicle Selects

n vehicle-Selects (AbD, AfC, AfP)

- Check each delivery at acceptor if it can be moved to provider (see previous slide)
- Determine responsible vehicle at provider for each potentially moved delivery according FiFo-rule
- Check all vehicles of provider at acceptor and if it is useful for each transformation AbD, AfP, AfC
  - (i) Empty travel, continuation after delivery of (ii) the most suitable own delivery and (iii) the best AP delivery that was determined before
- Estimate effort for each transformation and select best
  - Availability at acceptor plant – the earlier the better
  - Malus in case of First Round-delivery
  - Prefer transformation with each truck type by increasing `fear.fifo_truckSelect_firstOfTypeBonus`
  - Bonus if additional load is possible with `fear.fifo_truckSelect_loadDoneBonus`
- Select best *n* transformations

ADDITONAL PARAMETERS

Additional Parameters

`optim`	`createFullNumberOfAPTrafos`	Configure if only two (default) or all (=theTransformationsPerPreSelect) AP-transformations are selected
`fear`	`shelfLifeViolations_areUndeliverable`	No proposals, if shelf life is exceeded, e.g., by travel duration
DG	`thePlusLoadPercentage`	Percentage of average delivery quantity used for Plusload → Small Plusloads potentially delivered with last regular load if vehicle capacity sufficient

CONCLUSION

Overview

- Name: FiFo or Transfer-Optimization
- FiFo-rule, i.e., first vehicle gets next load
- Orders are broken down into deliveries according to the defined load-spacing
- Load-spacing and sequence of deliveries are considered
- Two-step approach with Plant Dispatch (starting point) and Transformations (optional)
- Various constraints like First Round-rules or Morning Roster
- Planning from left to right

Nice to know:

- Only signed on vehicles or vehicles with a SAS are considered, while FAS and fixed deliveries are not considered (except configured by `global.fifoStartPlanningAfterFixedProposals`)
- No multi-shift functionality and planning horizon is one day with hard break at day-change
- Costs are configurable (see parameters)
- Debug-output can be switched on: `msconfig -v"fear.log_optimizer_enable=1"`

DISCUSSIONS AND QUESTIONS

INFORM